Réalisation des prémices d'un noyau

Chaput Emmanuel

Version 0.00000001

Résumé

Ce document n'est constitué pour le moment que de quelques notes que j'ai pu prendre en essayant de réaliser le début d'un truc qui pourrait vaguement laisser penser que j'essaye de réaliser un noyau de système d'exploitation.

Ces notes me sont purement personelles et je vous invite donc, qui que vous soyez, à arréter de lire ça ! Vous êtes encore là ? C'est pas sympa ...

Table des matières

1 Introduction **3**

2 Organisation générale **3**
2.1 Structure du code . . . 3
2.1.1 Le code "utilisateur" . . . 4
2.1.2 Le code du noyau proprement dit . . . 4
2.2 Configuration . . . 4
2.2.1 Le fichier `include/config.h` . . . 4
2.2.2 Le fichier `make.conf` . . . 5
2.2.3 Le fichier `taille.conf` . . . 5
2.3 Organisation de ce document . . . 5

3 Les outils de développement **6**
3.1 La compilation . . . 6
3.1.1 L'assembleur . . . 6
3.1.2 Le compilateur C . . . 6
3.1.3 La création du noyau . . . 6
3.2 L'exécution . . . 6
3.2.1 L'émulateur Bochs . . . 6
3.2.2 L'émulateur QEMU . . . 6
3.3 Le débugage "interne" . . . 6
3.3.1 La fonction `printk_debug` . . . 6
3.3.2 La macro `assert` . . . 7
3.3.3 Le handler `handlerPanique` . . . 7
3.3.4 Le fichier `noyau/symbols` . . . 7
3.4 Le débugage "externe" . . . 7
3.4.1 Exécution . . . 7
3.4.2 Affichage des registres . . . 7
3.4.3 Affichage des variables . . . 7
3.4.4 Désassemblage du code . . . 8

4 Le boot **8**
4.1 Construction du fichier de démarrage 8
4.2 Liens entre les trois éléments 8
4.3 Le secteur de boot 9
4.4 État de la mémoire après le chargement 9
4.5 L'initialisation 9
4.5.1 Passage des informations au noyau 10
4.5.2 Détection de la mémoire 10
4.5.3 Détection du processeur 10
4.5.4 Démarrage du noyau (`noyau/main.c`) 10

5 L'affichage (`lib/console.c lib/printk.c`) **11**
5.1 Gestion de l'écran : la console 11
5.2 Plus de souplesse avec les consoles virtuelles 11
5.3 Mise en place d'un journal 12
5.4 Un premier `printk()` 12

6 Gestion des interruptions (`interruptions.c interBasNiveau.nasm`) **12**
6.1 Définition "bas niveau" d'un handler 12
6.2 Définition d'un handler en C 13
6.3 Insertion d'un handler dans l'IDT 13
6.4 Initialisation de la table 13
6.5 Le timer 14
6.6 Le clavier 14

7 Les appels systèmes **14**
7.1 Définition des éléments communs noyau/application 15
7.2 Définition de l'interface 15
7.3 Écriture de l'appel système 15
7.4 Enregistrement de l'appel système 16

8 Un premier `init` **16**

9 Gestion des tâches **16**
9.1 Définition d'une tâche 17
9.1.1 Champs liés au noyau 17
9.1.2 Champs liés au processeur 17
9.1.3 Les descripteurs de tâche Intel 17
9.2 Création d'une tâche 17
9.3 Basculement vers une tâche 17

10 Gestion de la mémoire **18**
10.1 La segmentation 18
10.1.1 Les descripteurs de segment 19
10.1.2 Les tables de descripteurs de segments LDT et GDT 20
10.2 Adresse linéaire et adresse physique 20
10.3 La mémoire virtuelle 21
10.3.1 Pagination 21
10.3.2 Gestion de la mémoire physique dans ManuX 21
10.3.3 Organisation de la mémoire virtuelle 23

11 La librairie du noyau **23**
11.1 Le journal 23
11.2 Le ramdisk 23

12 Les outils de synchronisation **23**
12.1 Opérations atomiques 23
12.2 Exclusions mutuelles 24

13 L'ordonnancement des tâches **24**

14 Le système de fichiers **24**

A Les appels système de ManuX **24**
A.1 creerNouvelleTache 24
A.2 basculerTache 24
A.3 ecrireConsole 24

1 Introduction

Ce document rassemble quelques notes prises durant ma tentative de développement de *ManuX* ("Misérable Approximation de Noyau Un*X"). Mon but en développant cette chose est uniquement de mieux comprendre les problèmes liés à l'implantation d'un système d'exploitation. Je n'ai en aucun cas comme objectif d'arriver à un OS utilisable à d'autres fins que tester des mécanismes systèmes.

De plus, je ne recherche absolument pas la performance dans le code que j'écris, mais plutôt la lisibilité et la compréhensibilité. Ca peut ne pas sauter aux yeux dans les parties de code encore en développement (c'est à dire partout pour le moment) . . .

J'essaie de plus de coder les choses aussi proprement que possible. Je trouve par exemple pitoyable des structures du genre

```
struct processus {
   int pid;
   ...
   struct processus * suivant; /* de la liste */
}
```

dans lesquelles on mélange allègrement le contenu (ici un processus) et le contenant (une liste par exemple) ; c'est ce que j'appelle le "syndrome du poisson pané".

Personnellement je définirai donc d'abord les caractéristiques du processus sans contrainte extérieure puis celle du contenant sans se préoccuper du contenu. Une telle façon de programmer me parait beaucoup plus structurée, simple et compréhensible ; de plus elle se prète à la mise en œuvre de structures "génériques", même si en C cela reste un peu du bricolage. Qui plus est, une telle programmation peut se faire sans nuire aux performances.

2 Organisation générale

Comme tout système digne de ce nom, Manux-32 définit un mode utilisateur et un mode noyau. Il n'utilise pour l'instant pas les différents modes fournis par le processeur, mais j'ai bon espoir pour que cela arrive bientôt.

2.1 Structure du code

Le code de ManuX est réparti dans quelques fichiers, eux-mêmes organisés dans quelques répertoires.

Au plus haut niveau, nous allons distinguer deux grandes parties que sont le "code noyau" et le "code utilisateur".

2.1.1 Le code "utilisateur"

Le code du monde utilisateur est confiné dans le répertoire `usr`. Les principaux fichiers de `usr` sont les suivants

`unistd.c` implante les principales fonctions (décrites dans `include/unistd.h`) permettant l'accès aux fonctions du noyau. Comme nous le verrons, ce sont donc surtout des appels systèmes.

`stdio.c` implante les principales fonctions (décrites dans `include/stdio.h`) permettant de réaliser des entrées/sorties.

`init.c` implante le code de la première tâche lancée automatiquement par le noyau lorsqu'il est initialisé.

La compilation du code utilisateur nécessite naturellement des fichiers inclus. Ils se trouvent dans le répertoire `usr/include`. Certains des fichiers, situés dans le répertoire `include/manux` sont en fait des copies depuis l'arborescence du noyau. Ils sont mis à jour au travers de la cible `usrinc` du `Makefile principal`. Il s'agit de

`config.h` qui décrit la configuration du noyau, il est donc important de pouvoir en disposer dans le mode utilisateur afin de savoir ce qui est disponible ;

`types.h` qui définit les types utilisés dans le noyau ;

`appelsystemenum.h` qui donne les numéros des appels systèmes ;

`string.h` est là pour offir au mode utilisateur des fonctions présentes dans le noyau (`memcpy()`,...). Il n'est pas nécessaire que ce soit le même, mais cela évite de tenir à jour deux fichiers !

2.1.2 Le code du noyau proprement dit

`lib`

`noyau`

`i386`

`outils`

`boot`

`sf`

`doc`

2.2 Configuration

La configuration se fait essentiellement au travers du fichier `manux/config.h`.

Certaines valeurs sont utilisées ailleurs que dans du C (dans du NASM, ou directement dans le `Makefile`) et une cible `make` est là pour générer un fichier `make.conf` qui contient ces variables. Pour qu'elle fonctionne, il est impératif que ces macros débutent par *MANUX*_

D'autres valeurs doivent être déterminées durant la phase de compilation (par exemple la taille du noyau). Elles sont donc calculées par le programme `outils/taillenoyau` et placées dans le fichier `taille.conf`.

2.2.1 Le fichier `include/config.h`

Il permet de choisir l'inclusion, ou non, de certines parties du noyau, et parfois de les paramétrer.

Configuration du fichier de boot

`MANUX_TAILLE_PAGE`

`MANUX_NOMBRE_PAGES_SYSTEME`

`MANUX_ADRESSE_DEBUT_TAS`

`MANUX_NB_MAX_FICHIERS`

`MANUX_CONSOLES_VIRTUELLES`

`MANUX_JOURNAL`

Configuration de l'adressage du noyau

`MANUX_TAILLE_PAGE`

`MANUX_NOMBRE_PAGES_SYSTEME`

`MANUX_ADRESSE_DEBUT_TAS`

Configuration de l'affichage

`MANUX_CONSOLES_VIRTUELLES`

`MANUX_JOURNAL`

Configuration du système de fichiers

`NB_MAX_FICHIERS`

2.2.2 Le fichier `make.conf`

2.2.3 Le fichier `taille.conf`

2.3 Organisation de ce document

Mon but est de décrire les choses de façon progressive, dans l'ordre dans lequel je les ai faites, et dans l'ordre dans lequel il me semble pertinent de les faire de sorte à construire progressivement un noyau qui fonctionne.

La section "*Les outils de développement*" décrit sommairement les outils nécessaires à la compilation, le débogage et le lancement de ManuX.

La section "*Le boot*" montre comment faire en sorte que notre code soit chargé en mémoire par le BIOS de la machine puis exécuté. C'est une partie assez technique, très spécifique au matériel ciblé, et peu intéressante. Je vous autorise à la passer. À la fin de cette partie, le code du noyau lui même commence à être exécuté, c'est donc là que les choses deviennent croustillantes ! Mais c'est également là que les ennuis commencent, ...

Afin de pouvoir voir ce que nous faisons, une première activité va consister à écrire du code pouvant afficher des choses à l'écran, nous pourrons enfin avoir notre "`Hello world !`" du noyau. Pour cela, nous allons créer une "console" et une fonction `printk` dans la section "*L'affichage*".

La section "*Les appels systèmes*" sera ensuite consacrée à mettre en place le mécanisme permettant aux applications de communiquer avec le noyau. Pour le moment, ce mécanisme n'est pas nécessaire, un simple appel de fonction permet cela, mais lorsque nous mettrons en place des mécanimes de protection, nous seronsbien contents d'avoir fait les choses proprement !

Dans la section "*Un premier* `init`", nous alons écrire un premier "code utilisateur"

Nous allons ensuite nous attaquer à un morceau assez conséquent : la gestion de la mémoire qui sera décrite dans la section "*La gestion de la mémoire*". Nous ferons dans ManuX les choses aussi simplement que possible.

3 Les outils de développement

3.1 La compilation

Observons les principaux éléments du `Makefile`.

3.1.1 L'assembleur

J'utilise NASM pour les quelques fichiers écrits en assembleur.

3.1.2 Le compilateur C

3.1.3 La création du noyau

Sans grande surprise, on compile avec

```
make
```

Différentes cibles sont créées

```
manux
run
clean
```

3.2 L'exécution

Le but n'est évidemment pas de faire tourner cette merveille sur du vrai matériel, mais plutôt dans un émulateur.

3.2.1 L'émulateur Bochs

J'utilisais initialement cet outil, mais les fichiers de configuration ne semblent plus convenir aux dernières versions de Bochs, il faudrait les mettre à jour ! Des volontaires ?

3.2.2 L'émulateur QEMU

C'est maintenant `qemu` que j'utilise, par exemple de la façon suivante

```
qemu-system-i386 -drive format=raw,file=manux,index=0,if=floppy -m 64M
```

C'est d'ailleurs la commande que lance

```
make run
```

3.3 Le débugage "interne"

Ce que j'appelle ici le débugage interne, c'est tous les outils que l'on peut utiliser dans le code lui-même pour essayer de comprendre ce qu'il s'y passe.

Nous avons pour le moment quelques outils de base, copieusement bugués eux-mêmes (!).

3.3.1 La fonction `printk_debug`

Elle permet de produire des messages d'erreur qui seront envoyés de façon sélective en fonction de la valeur de *masqueDebugage* qui doit être construite dans `manux/debug.h`.

3.3.2 La macro `assert`

Voir `manux/debug.h`

3.3.3 Le handler `handlerPanique`

3.3.4 Le fichier `noyau/symbols`

Il va surtout nous servir pour aider au débugage externe, mais il est généré à la compilation du noyau.

3.4 Le débugage "externe"

Une technique pour débuguer est d'utiliser `qemu` couplé avec `gdb`. Pour cela, je lance par exemple `qemu` de la façon suivante

```
qemu-system-i386 -drive format=raw,file=manux,index=0,if=floppy
-rtc
base=localtime -m 64M -gdb tcp::1234  -S
```

Je lance ensuite `gdb` dans un autre terminal avec un `.gdbinit` qui contient :

```
target remote :1234
dir noyau
dir lib
file noyau/noyau.elf
```

La commande *`dir`* permet de dire à `gdb` où trouver les fichiers sources et la command *`file`* lui permet de consulter le binaire du noyau pour y trouver l'adresse des fonctions et variables (ici en fait, c'est une autre compilation du noyau au format ELF!).

3.4.1 Exécution

Quelques commandes de base pour l'exécution de code dans `gdb`

r `un` permet de lancer l'exécution d'un programme (inutilisable ici étant donné notre fonctionnement : on débogue à distance);

c `ontinue` permet de poursuivre l'exécution jusqu'au prochain point d'arrêt/bug/fin/...;

s `tep` avance d'une instruction dans le code;

n `ext`

b `reakpoint fonction`

b `reakpoint fichier:ligne`

3.4.2 Affichage des registres

Il peut être intéressant de consulter le contenu des registres

`info registers` donne le contenu des principaux registres.

3.4.3 Affichage des variables

`Gdb` permet également de consulter le contenu des variables :

`p toto` affiche le contenu de la variable `toto`.

3.4.4 Désassemblage du code

Une fonctionnalité intéressante est de pouvoir observer le code assembleur, ce qui peut permettre de comprendre certains comportements étranges, tels que les conséquences de choix d'optimisation du compilateur :

`disassemble fonc` affiche le code assembleur de la fonction `fonc`.

4 Le boot

Nous allons ici voir comment en arriver à un noyau présent dans la mémoire de la machine, prêt à être exécuter. Nous allons ici nous fonder sur une situation "à l'ancienne" avec un PC doté d'un lecteur de disquettes.

Nous allons donc devoir mettre en œuvre le boot du noyau dans cette configuration. Pour cela, nous allons construire un fichier qui sera placé sur une disquette à partir de laquelle notre système pourra démarrer. Ntons que nous utiliserons ici un émulateur comme Qemu auquel nous donnerons le chemin vers ce fichier et à qui nous demanderons de démarrer depuis cette disquette.

Observons la structure de ce fichier puis regardons comment en construire chaque partie.

4.1 Construction du fichier de démarrage

Afin de permettre un démarrage "à l'ancienne" par un boot sur disquette du PC, nous allons donc construire une image de disquette dont la structure est décrite par la figure 1.

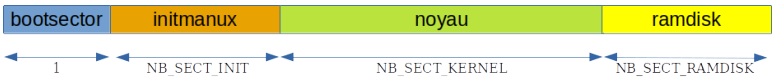

FIGURE 1 – Le fichier de démarage

Ce que j'appelle ici le boot du système se passera donc en deux temps. D'abord le secteur de boot est chargé en mémoire par le BIOS qui lui donne la main.

Le secteur de boot s'occupe alors de charger en mémoire le code d'initialisation ainsi que le code du noyau. Il donne alors à son tour la main à l'initialisation. Ce second élément réalise quelques opération puis démarre enfin le noyau.

Si le secteur de boot et le code d'initialisation sont deux éléments distincts, c'est pour deux raisons principales

— d'une part le secteur de boot ne dispose que de 512 octets, si bien qu'il doit être réduit au minimum ;
— chacun de ces deux éléments à un rôle bien spécifique (charger le code en mémoire et initialiser le système).

4.2 Liens entre les trois éléments

Les trois parties de codes évoquées ici (le secteur de boot, le code d'initialisation et le noyau) ne sont pas liées entre elles (par une édition de liens). Les appels de fonction, partages de variables, ... sont alors à éviter car peu pratiques !

Concrètement, il y a surtout deux "appels de fonction" à réaliser :

— le secteur de boot doit lancer le code d'initialisation ;
— le code d'initialisation doit démarrer le noyau.

Pour cela, on va utiliser le fait que tous les éléments sont placés à des adresses connues et faire un saut à ces adresses.

4.3 Le secteur de boot

Lorsqu'on allume un PC normalement configuré avec une disquette dans le premier lecteur, le BIOS charge le premier secteur de la diquette en mémoire et l'exécute. En fait, il ne fait cela que si les deux derniers octets de ce secteur contiennent la valeur magique `0xAA55`. Pour être plus précis, c'est à l'adresse `0x7C00` que ce secteur est chargé.

Un secteur étant composé traditionnellement de 512 octets, on ne peut pas faire des folies avec le secteur de boot ! On va donc se contenter de charger en mémoire d'autres secteurs du disque. Pour cela, on peut utiliser le BIOS et plus particulièrement l'interruption 13h qu'il nous fournit pour manipuler les lecteurs.

Le secteur de boot de ManuX est codé dans le fichier *`boot/bootsector.nasm`* et il est donc écrit en NASM. Il est très simple et se contente d'utiliser le BIOS pour charger le noyau en mémoire puis de faire un saut au début de la mémoire ainsi initialisée.

Il n'est pas très robuste, en particulier sur la taille attendue du noyau et sur l'adresse à laquelle le placer !

Concrètement, ce chargement se fait en deux phases :

— Chargement du code d'initialisation : c'est le code qui va avoir pour rôle d'initialiser le système. Ce code est écrit en assembleur dans le fichier *`boot/init-manux.nasm`*.

— Chargement du code du noyau : c'est tout le code de ManuX qui est ici chargé en mémoire. Il sera exécuté par le code d'initialisation.

Après ces deux chargements, le code du secteur de boot exécute le code d'initialisation.

Je ne décris pas plus ce code qui ne présente pas un intérêt profond, les plus curieuses et curieux n'ont qu'à lire le code !

4.4 État de la mémoire après le chargement

L'état de la mémoire à la fin du boot est décrite par la figure **??**.

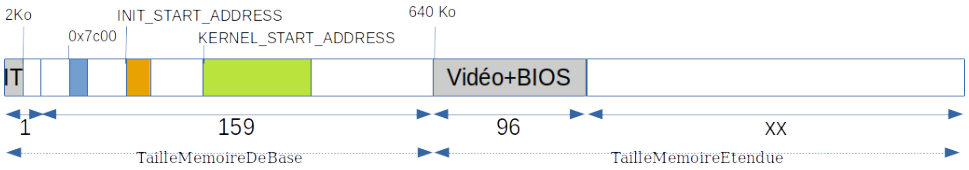

FIGURE 2 – État de la mémoire après le boot

Une fois qu'il a terminé son travail, le code du secteur de boot donne la main au code d'initialisation.

4.5 L'initialisation

Le "programme" d'initialisation est traditionnellement consacré à la détection et à l'initialisation (original, vu son nom !) du matériel. Sur un PC de base, on peut là encore se faire aider du BIOS.

En ce qui concerne ManuX, cette phase d'initialisation va profiter du fait qu'elle n'a pas les mêmes limites de taille que le secteur de boot (un seul secteur comme le nom au singulier le suggère). On va donc pouvoir y réaliser certaines actions un peu plus confortablement, comme :

— Vérifier la mémoire disponible (cette information sera passée au noyau).

— Charger en mémoire un ramdisk qui nous permettra de jouer avec des systèmes de fichiers sans avoir besoin d'implanter la gestion de matériel.

Voyons rapidement quelques points.

4.5.1 Passage des informations au noyau

La plupart des informations recueillies par l'initialisation peuvent se révéler particulièrement intéressantes pour le noyau. Il est donc important de se définir un mécanisme permettant de faire passer ces informations depuis la phase d'initialisation jusqu'au noyau.

Pour cela, nous déclarons la "fonction principale" du noyau comme prenant en paramètre l'adresse d'une structure dans laquelle seront enregistrées ces différentes informations. Nous aurons donc par exemple la définition suivante

```
typedef struct _InfoSysteme {
   uint16 memoireDeBase;
   uint16 memoireEtendue;
   /* ... */
} InfoSysteme;

void _start(InfoSysteme * infoSysteme)
{
   /* ... */
}
```

Naturellement, pour que tout cela fonctionne, il est nécessaire, à la fin de la phase d'initialisation, d'empiler l'adresse de la structure dans laquelle les informations ont été enregistrées. Voici donc par exemple à quoi peuvent ressembler les dernières lignes de la phase d'initialisation :

```
; Calcul de l'adresse des infos Système
;-------------------------------------
mov   eax, 0                    ; C'est à cette adresse que
mov   ax, MANUX_INIT_SEG_16     ; sont actuellement les infos
shl   eax, 4                    ; Il faut transformer
add   eax, InfoSysteme          ; en adresse "flat"
push  eax

; Et c'est parti, on saute sur le noyau !
;----------------------------------------
mov   eax, MANUX_KERNEL_SEG_16
shl   eax, 4
call  eax
```

Notons que, bien sûr, aucun contôle de type ne peut être fait entre les deux phases et qu'il est donc important que le programmeur définisse de façon cohérente la structure C et la zone mémoire gérée en assembleur.

4.5.2 Détection de la mémoire

4.5.3 Détection du processeur

4.5.4 Démarrage du noyau (`noyau/main.c`)

La dernière action du programme d'initalisation est donc l'exécution de la fonction `_start()`. Comme je l'ai dit précédemment, il ne s'agit pas d'un appel de

foncion C classique, puisque les deux morceaux de code ne sont pas liés entre eux, mais d'un saut à l'adresse de la fonction (le tout après avoir empilé les paramètres).

Ceci termine donc la description de toute la quincailleire qui nous permet d'arriver au début de l'exécution du code du noyau. Nous allons commencer la description de ce dernier par la mise en place des outils de base dont il aura besoin.

4.6 Utilisation de *Multiboot*

Nous allons ici utiliser la spécification *Multiboot* définie par la *Free Software Foundation* qui va nous permettre d'utiliser les services d'outils tels que GRUB afin de grandement simplifier le chargement du noyau de ManuX.

Cela nous permettra également d'utiliser des outils un peu plus modernes qu'un lecteur de disquettes et, par voie de conséquence, de faire fonctionner ManuX sur autre chose que des machines virtuelles ou hors d'age !

5 L'affichage (`lib/console.c lib/printk.c`)

Une des premières choses à faire, afin de nous simplifier la vie pour la suite du développement, est de se doter d'un service minimal d'affichage. Nous allons donc mettre en place un outil très simple fondé sur ce que nous founit le matériel.

Nous allons pour cela procéder en trois étapes.

— Dans un premier temps, nous allons écrire le code permettant un accès simple à l'écran et une gestion minimale de ce dernier dans un module "*Console*". Cet outil est suffisant pour que notre noyau soit capable de nous donner un signe de vie à l'écran. Mais nous n'allons pas nous arrêter en si bon chemin !

— Nous construirons là-dessus un "journal" du noyau, c'est-à-dire un outil permettant de rassembler et d'organiser les messages émis par les différents sous-systèmes du noyau. Ils pourront être affichés sur une console, ou conservés pour être stockés/traités d'une autre façon.

— Nous pourrons alors écrire une fonction `printk` utilisable partout dans le noyau et chargée de mettre en forme des messages riches et de les envoyer dans le journal.

L'avantage d'une telle structure est que nous pourrons la faire évoluer. Nous pourrons par exemple créér des consoles virtuelles fournissant le même service que notre outil de gestion de l'écran. Il sera très simple de les intégrer dans le noyau.

5.1 Gestion de l'écran : la console

Il se trouve que l'écran d'un PC peut être utilisé en mode textuel comme un simple tableau de caractères. Ce tableau est situé à une adresse bien connue a et contient l lignes de c caractères. Chaque caractère est codé sur deux octets : l'un définit le code ASCII du caractère affiché et l'autre la couleur du texte et du fond.

Les fichiers `manux/console.h` et `lib/console.c` implantent un outil de gestion de base de la console.

On construit ici en particulier la fonction suivante, chargée d'afficher sur l'écran une chaîne de caractères passée en paramètre :

```
void afficherEcran(char * msg);
```

Elle prend en paramètre une chaîne de caractères "classique" (terminée par un caractère nul) et l'affiche à l'écran sans aucune mise en forme.

La gestion de la console est une des toutes premières choses initialisées par le noyau, de sorte à pouvoir faire coucou dès que possible.

5.2 Plus de souplesse avec les consoles virtuelles

Lorsque nous allons mettre en place la notion de tâches, il sera intéressant d'attribuer à chacune d'entre elles une console virtuelle potentiellement différente des autres. Nous pourrons ainsi basculer de l'une à l'autre pour mieux distinguer ce qu'affiche chaque tâche.

5.3 Mise en place d'un journal

Nous allons définir un mécanisme de journal permettant de conserver les messages transmis par le noyau. Pour le moment, notre journal se contente de transmettre ces messages sur l'écran physique. Lorsque nous aurons mis en place des outils de gestion mémoire, nous pourrons étoffer ce mécanisme de journal.

Le journal est implanté dans les fichiers *manux/journal.h* et *lib/journal.c*. Il propose essentiellement la fonction suivante

```
void journaliser(char * message);
```

Elle utilise la fonction *afficherEcran()* pour envoyer le message à l'écran.

Évidemment, le journal est initialisé juste après la console.

5.4 Un premier `printk()`

Grâce à notre console , l'écriture de `printk()` est assez simple, puisque cette fonction n'a plus qu'à se préoccuper de créer une chaîne de caractères et de la transmettre au journal (ou directement à la console si on n'utilise pas le journal).

Elle est mise en œuvre dans les fichiers *manux/printk.h* et *noyau/printk.c*

La fonction essentielle est bien sûr :

```
void printk(char * format, ...);
```

Dont le rôle est de mettre en forme une chaîne de caractèreqs qu'elle envoie ensuite à la console ou au journal.

6 Gestion des interruptions (`interruptions.c interBasNiveau.nasm`)

Sur le 386, une table (l'IDT pour *Interrupt Descriptor Table*) définit le comportement à adopter pour chacune des 256 interruptions et exceptionns. Cette table est enregistrée grâce à l'instruction *lidt*.

Observons ici comment tout cela est mis en place dans ManuX.

Nous ne pouvons pas ici faire l'impasse sur un peu d'assembleur, si bien que les fonctions "bas niveau" seront implantées dans les fichiers `manux/i386/interBasNiveau.h` `i386/interBasNiveau.nasm`. Tout le reste sera en C dans `manux/interruptions.h` et `i386/interruptions.c`.

C'est la fonction C *initialiserIDT()* qui est invoquée par *main()* pour initialiser la table.

6.1 Définition "bas niveau" d'un handler

Voici par exemple comment est écrit un handler pour l'interruption 7 (mais c'est sans intérêt ici) dont le rôle est d'invoquer une fonction C nommée *handlerPanique*.

Notez que ce handler empile tous les registres ainsi que le numéro de l'interruption concernée.

```
; Un handler qui va afficher un message
;-------------------------------------
stubHandlerPanique 7

        pusha
        push dword 07h

        call handlerPanique

        add esp, 4
        popa

        iret
```

Vous trouverez également un handler qui ne fait rien :

```
; Un handler qui ne fait rien ...
;-------------------------------
stubHandlerNop :
        iret                        ; On revient ...
```

Notons que lors d'une interruption, le microprocesseur empile les valeurs de `EFLAGS`, `CS` et `EIP` (voir par exemple [**?**] page 159).

Voyons maintenant comment écrire la suite plus confortablement en C.

6.2 Définition d'un handler en C

Nous pouvons par exemple écire la fonction *handlerPanique* dont le rôle est d'afficher un message permettant de déterminer la cause de l'interruption :

```
void handlerPanique(uint32 itNum, TousRegistres registres,
                    uint32 eip, uint32 cs, uint32 eFlags)
{
   ...
}
```

Nous la définissons avec les paramètres qui sont, dans l'ordre inverse, les valeurs empilées par le processeur lors de l'interruption puis par le handler "bas niveau" décrit ci-desssus.

La structure *TousRegistres*, décrite dans `manux/types.h`, permet de manipuler simplement l'ensemble des registres.

6.3 Insertion d'un handler dans l'IDT

La procédure suivante initialise une entrée dans l'IDT :

```
void positionnerHandlerInterruption(IDT idt, int i, Handler handler)
/*
 * Affectation du handler de l'interruption i
 */
{
   idt[i].itg.offsetFaible = ((uint32)handler & 0xFFFF);
   idt[i].itg.offsetFort = ((uint32)handler >> 16);
   idt[i].itg.parametres = 0x8E00;
```

```
   idt[i].itg.selSegment = SELECTEUR_SEGMENT_CODE;
}
```

Elle sera donc appelée par la procédure d'initialisation de l'IDT pour chacune des interruptions.

6.4 Initialisation de la table

Nous avons maintenant tous les éléments pour construire notre IDT. C'est le rôle de la fonction C `initialiserIDT()`.

Nous y positionnons donc tous les handlers sur le handler minimaliste qui ne fait rien :

```
/* Comportement par défaut : on ne fait rien ! */
for (i = 0; i < 256; i++) {
   positionnerHandlerInterruption(idt, i, stubHandlerNop);
}
```

Nous allons ensuite définir un handler un peu plus utile pour les interruptions que peut nous générer le matériel

```
positionnerHandlerInterruption(idt, 0, stubHandlerPanique_0);
positionnerHandlerInterruption(idt, 1, stubHandlerPanique_1);
...
```

Nous pouvons enfin définir les handlers plus spécifiques

```
/* Le handler du timer */
positionnerHandlerInterruption(idt, intTimer, stubHandlerTimer);

/* Le handler du clavier */
positionnerHandlerInterruption(idt, intClavier, stubHandlerClavier);
```

6.5 Le timer

6.6 Le clavier

Le clavier est géré dans `manux/clavier.h` et `clavier.c`. Il s'agit essentiellement d'une fonction d'initialisation et du handler de l'interrruption correspondante.

Ce handler doit bien entendu être déclaré dans `i386/interBasNiveau.nasm`.

7 Les appels systèmes

Les appels systèmes constituent une part importante de l'interface entre le noyau et les applications. Dans ManuX, le choix a été fait d'implanter ce mécanisme au travers d'une interruption. C'est une technique classique (utilisée par exemple par Linux).

Leur mise en œuvre passe part plusieurs étapes que je vais détailler ici. L'objectif d'un appel système est d'offrir à une application utilisateur un service qui ne peut être rendu que part le noyau. Pour des raisons de confort d'utilisation, ces services sont rendus sous la forme de fonctions classiques, comme une librairie traditionnelle. Pour en arriver là, nous pouvons distinguer quatre étapes

— L'application utilise une fonction C pour invoquer un appel système. Le code de cette fonction doit donc être construit de sorte à préparer puis mettre en œuvre la sollicitation du service auprès du noyau (par le biais d'une interruption ici).

— L'interruption est déclanchée et le code du handler de cette interruption est exécuté.
— Le handler va devoir déterminer quelle est la fonction du noyau qui doit être utilisée pour rendre le service voulu à l'application.
— La fonction en question va être exécutée et le service fourni à l'application.

Nous allons maintenant décrire la mise en œuvre de ces étapes en prenant comme exemple un appel système qui nous permettra d'envoyer un message sur la console. Nous pourrons ainsi écrire un `printf` pour les applications !

7.1 Définition des éléments communs noyau/application

Le fichier `manux/appelsystemenum.h` donne en particulier les numéros des différents appels systèmes. Il doit donc absolument être commun au noyau et aux "applications".

7.2 Définition de l'interface

Ce que j'appelle interface est la signature de la fonction directement utilisable dans une application pour invoquer un appel système.

Cette interface est donc décrite côté applications, dans les fichiers de `usr`.

Des macros ont été créées dans le fichier `manux/appelsysteme.h` pour simplifier une telle définition. Ce sont par exemple les "fonctions" *appelSysteme0*, *appelSysteme1*,...

On utilisera donc une telle macro chaque fois qu'il sera nécessaire de définir un nouvel appel système. C'est ainsi que dans le fichier *usr/include/unistd.h* on trouve par exemple :

```
appelSysteme2(NBAS_AFFICHER, int, afficherConsole, void *, int);
```

Cette ligne défini donc la fonction *afficherConsole()* qui va permettre à une application d'envoyer un message sur la console.

7.3 Écriture de l'appel système

La partie la plus importante est naturellement d'écrire le code de l'appel système. Celui-ci consiste simplement en une fonction C classique (qui en utilise éventuellement d'autres, naturellement). La seule contrainte étant que si cette fonction a des paramètres, le premier d'entre eux (qui ne présente aucun intéret pour le programmeur) doit être de type *ParametreAS*.

Ce paramètre sert uniquement a "recadrer" les paramètres suivants dans la pile suite aux décalages induits dans celle-là par les appels successifs à l'interface de l'appel système, à l'interruption, et à l'appel système lui-même d'une part, et à la sauvegarde des registres d'autre part.

Observons par exemple le code permettant la mise en œuve de l'appel système *ecrireConsole*. Il se trouve dans le fichier `lib/console.c` :

```
/*
 * L'implantation de l'AS ecrireConsole
 */
int sys_ecrireConsole(ParametreAS as, void * msg, int n)
{
   assert(tacheEnCours != NULL);
   Console * cons = tacheEnCours->console;

   assert(cons != NULL);
```

```
    afficherConsoleN(cons, msg, n);

    return n;
}
```

Bien sûr, cette fonction utilise le code de manipulation de la console déjà écrit.

7.4 Enregistrement de l'appel système

L'enregistrement de l'appel système dans le système d'exploitation est réalisé par une invocation de la fonction suivante, définie dans `appelsysteme.h` et codée dans `noyau/appelsysteme.c` :

```
int definirAppelSysteme(int num, void * appel);
```

où

est le numéro de l'appel système enregistré ;

est la fonction de traitement de cet appel système.

Le travail de cette fonction est donc simplement de placer dans la table des appels systèmes (*`vecteurAppelsSysteme`*) la fonction de traitement de cet appel système.

C'est donc ici la "partie noyau" qui est réalisée, c'est-à-dire la définition des actions à mener lorsque l'appel système est invoqué.

Nous trouvons par exemple dans la fonction *`initialiserAppelsSysteme()`* du fichier `noyau/appelsysteme.c` la ligne de déclaration de notre appel système *`ecrireConsole`* :

```
void initialiserAppelsSysteme()
{
    ...

    /* Envoyer une chaîne de caractères sur la console */
    definirAppelSysteme(NBAS_ECRIRE_CONS, sys_ecrireConsole);

    ...
}
```

8 Un premier `init`

Nous avons désormais tout ce qu'il nous faut pour mettre en place du "code utilisateur", c'est-à-dire un code qui ne fait pas partie du noyau, mais qui consitue une application utilisant les services du noyau au travers des appels système.

Attention, comme mentionné précédemment, les "applications" s'exécutent pour le moment dans le même mode que le noyau.

9 Gestion des tâches

Nous disposons désormais du minimum permettant d'exécuter quelques instructions et d'en voir les conséquences à l'écran. Je vous propose donc de mettre en place la notion de *tâche* qui nous permettra de faire avancer "en même temps" plusieurs activités. ManuX devient donc à partir de maintenant un système multitâche.

Attention, pour le moment, ce ne sera que du multitâche collaboratif. En effet, nous n'allons pas encore mettre en place du multitâche *préemptif*, ce qui signifie que une tâche ne rendra la main (enfin, le processeur) que lorsqu'elle en aura envie !

De plus, nous ne nous intéressons pas non plus à l'ordonnancement, et mettrons en place un simple *round robin*.

9.1 Définition d'une tâche

La structure d'une tâche est définie dans le fichier `manux/tache.h` au travers d'une struture *`Tache`*. Afin de nous simplifier la vie, nous allons utiliser dans ManuX l'aide proposée par les processeurs Intel [1].

De ce fait, la structure *`Tache`* contient des champs liés au processeur et des champs purement liés au noyau.

9.1.1 Champs liés au noyau

9.1.2 Champs liés au processeur

- *`IntelTSS tss`* qui permet, comme son nom laisse à penser, de stoquer les informations relatives à l'état de la tâche au niveau du processeur (TSS pour *Task State Segment*). On trouvera des informations sur ce champs dans les documentations Intel. Dans ManuX, une initialisation minimale est faite de cette structure lors de la création d'une tâche dans *`creerTache`*.
- *`uint16 indiceTSSDescriptor`* représente l'indice, dans la GDT, du TSS *Descriptor* de la tâche. C'est par un **`ljmp`** vers ce champs que l'on va activer la tâche.
- *`DescriptorTable * ldt`* est la table locale de decripteurs associée à cette tâche.

Étant donnée la gestion limitée de la mémoire à ce niveau du système, les structures définissant une tâche sont placées dans une même page mémoire et organisées comme représenté sur la figure 3.

FIGURE 3 – Structure d'une page de tâche

9.1.3 Les descripteurs de tâche Intel

Le *Task State Segment* (TSS) des processeurs Intel permet de stoquer et de manipuler les informations relatives à une tâche. Dans ManuX, une structure est définie permettant de décrire dans le noyau le TSS de chaque tâche : la structure *`IntelTSS`* est définie dans *`manux/tache.h`*

9.2 Création d'une tâche

9.3 Basculement vers une tâche

En ce qui concerne le basculement d'une tâche à l'autre, nous allons profiter des mécanismes fournis par les processeurs Intel au travers de l'instruction `ljmp`.

1. Les systèmes récents, pour des raisons de performances, n'utilisent pas ces fonctionnalités.

L'*Indirect far JMP* permet en effet de faire un basculement de tâche lorsque l'argument qui lui est passé, sur 48 bits (oui madame, 6 octets !) est constitué d'un sélecteur de segment (sur 16 bits) qui référence un TSS puis d'un *offset* (qui ne sera pas utilisé pour le coup !).

Bref, le basculement d'une tâche vers une autre se résume à une instruction de saut !

10 Gestion de la mémoire

La gestion de la mémoire est une fonction essentielle d'un système d'exploitation. Nous allons dans un premier temps jeter un coup d'œuil rapide à la gestion de la mémoire vue par Intel.

Il y a plusieurs concepts à intégrer. Cela peut sembler complexe, mais en prenant un peu de temps, vous vous rendrez compte qu'il n'y a rien de méchant !

Le premier est la *segmentation*. Pour faire simple, imaginez-vous fin des années 70 avec un microprocesseur dont les registres sont sur 32 bits. Comment adresser plus de 64 Ko ? La segmentation consiste à utiliser deux registres pour composer une adresse de plus de 32 bits. Nous allons voir que ce concept a été enrichi et utilisé à d'autres fins.

Le second est la *pagination*. L'idée ici est de découper la mémoire en *pages* de taille fixe. Chaque page pourra être dotée de propriétées qui lui sont propre. La pagination va nécessiter la mise en place d'un certain nombre de tables d'indirection, ce qui peut sembler un peu lourd, mais qui va en particulier permettre de la mémoire virtuelle, de l'isolation entre les tâches, ...

10.1 La segmentation

Essayons de comprendre ce qui se passe lorsqu'une instruction du processeur doit manipuler une adresse. Nous nous en tiendrons ici à l'architecture des Intel 386.

Imaginons par exemple une instruction qui doit aller lire ou écrire un octet en mémoire ou encore une instruction qui effectue un saut vers une nouvelle zone de code. Une telle instruction va donc exprimer explicitement une adresse, appelée *adresse logique*. Cette adresse logique est composée de deux parties : un *segment*, et un *offset*.

Le segment est une partie de la mémoire, et l'offset un déplacement dans le segment, comme l'illustre la figure 4. On peut donc voir ça comme une découpe de la mémoire en zones (ou segments) logiques et/ou comme une façon d'accéder à des adresses plus élevées que ce que permet la taille des registres.

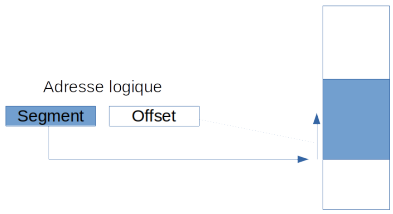

FIGURE 4 – Adresse logique

Le segment est contenu dans un registre qui peut être fourni explicitement ou implicitement par l'instruction. Il donne donc l'adresse de la zone mémoire que l'on appelle un segment. Sauf que c'est malheureusement un peu plus complexe

que ça ! Un segment n'est en effet pas décrit uniquement par son adresse (et quand bien même ! On a dit qu'un registre ne pouvait pas contenir une adresse entière).

En fait, le registre en question permet d'accéder à un *descripteur de segment*. C'est évidemment ce dernier (que nous allons décrire par la suite) qui donne, entre autres, l'adresse du segment. La figure 5 illustre cette indirection.

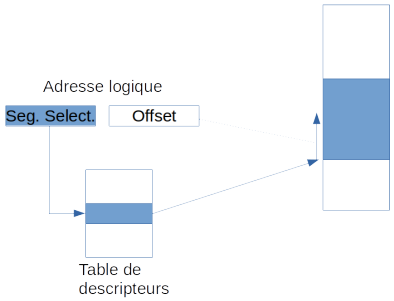

FIGURE 5 – Adresse logique et sélecteur de segment

Dans les versions antérieures de ses processeurs (et dans le mode "réel" du 386), la mémoire est découpée logiquement en *segments*. Une adresse est alors désignée par un couple composé d'un *registre de segment* et d'un *registre d'offset*.

Concrètement, la taille d'un segment est de 64 Ko (puisque le registre d'offset est sur 32 bits).

À partir du 286 (si je ne me trompe pas), en mode dit "protégé", un registre de segment devient une référence vers un *descripteur de segment*. Ce dernier permet une gestion beaucoup plus évoluée de la mémoire.

Les descripteurs de segment sont rassemblés dans une table, la GDT (**G***lobal* **D***escriptor* **T***able*) ou la LDT (**L***ocal* **D***escriptor* **T***able*). Un registre de segment représente alors simplement le décalage d'un descripteur dans la table.

Lorsque l'on utilise un registre de segment, il contient donc en fait l'adresse (dans la LDT ou dans la GDT) d'un descripteur de registre. Ce descripteur contient lui-même l'adresse du segment.

10.1.1 Les descripteurs de segment

La figure 6 montre la structure d'un descripteur de segment.

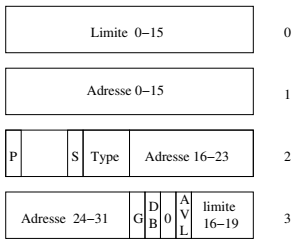

FIGURE 6 – Structure d'un descripteur de segment

Dans ManuX, la fonction suivante crée un descripteur de segment et l'ajoute dans la GDT ou la LDT passée en paramètre :

```
int setDescripteurSegment(DescriptorTable * dt,
                          uint32 adresse, uint32 limite,
                          uint8 type,
```

```
uint8 gd0a)
```

Pour cela, elle initalise une structure de type `DescSegment` définie dans `include/i386/segment.h` de la façon suivante

```
typedef struct _DescSegment {
   uint16 limiteFaible;
   uint16 baseFaible;
   uint8  baseInter;
   uint8  type;
   uint8  limiteFort;
   uint8  baseFort;
} DescSegment;
```

10.1.2 Les tables de descripteurs de segments LDT et GDT

La GDT est donc la table globale contenant les descripteurs de segment du système.

Dans ManuX, elle est initialisée par la fonction suivante

```
void initialiserGDT();
```

Pour le moment, cette fonction est la plus simple possible, elle crée quatre descripteurs de segments

— un descripteur nul, obligatoire ;
— un descripteur pour le segment de code, occupant les 4 Go de base ;
— un descripteur pour le segment de données, occupant les 4 Go de base ;
— un descripteur pour le segment de pile, occupant les 4 Go de base.

Les trois segments se recouvrent donc.

La LDT est une table similaire à la GDT mais spécifique à une tâche. Dans ManuX, pour le moment, elle est initialisée, lors de la création de chaque tâche, comme une copie de la GDT.

10.2 Adresse linéaire et adresse physique

C'est finalement relativement simple, non ? Alors continuons, . . .

L'adresse ainsi obtenu est une adresse qualifiée de *linéaire*. La structure d'une adresse linéaire est donnée dans la figure **??**.

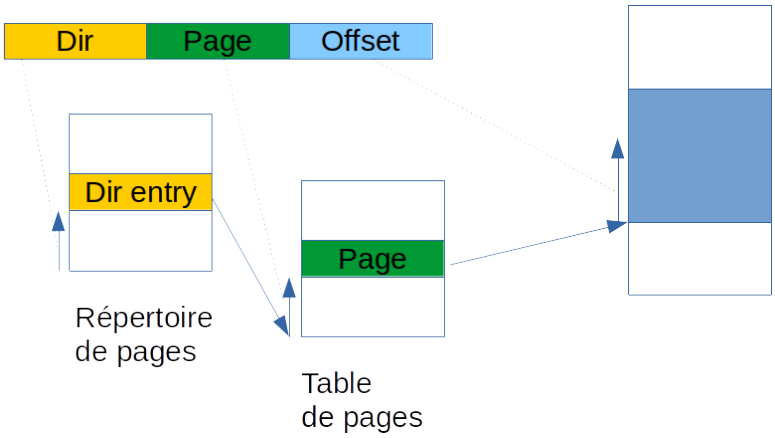

FIGURE 7 – Structure d'une adresse linéaire.

Les bits de poids forts (`Dir`) donnent la position dans le répertoire de page de l'adresse d'une table de pages. Les bits suivants (`Page`) donnent la position dans dans cette table de l'adresse d'une page en mémoire. Les derniers bits (`Offset`) donnent alors la position recherchée dans cette page.

Bien entendu, toutes les tables évoquées ici sont stoquées en mémoire.

10.3 La mémoire virtuelle

Les processeurs Intel offrent depuis le 386 un mécanisme appelé *pagination* permettant de définir un espace d'adressage *virtuel* qui est réparti sur un espace d'adressage physique. Une partie de cet espace physique peut être située ailleurs qu'en mémoire centrale, ce qui permet de construire une mémoire virtuelle plus grande que la mémoire réellement disponible.

De plus, la gestion de la pagination peut être spécifique à chaque tâche, ce qui permet de construire un système multitâche dans lequel chaque tâche a une vision de l'espace mémoire qui lui est propre.

Il y a donc deux problèmes à résoudre, que sont l'organisation de la mémoire physique (et sa répartition entre les différentes tâches) et l'organisation de la mémoire virtuelle vue par une tâche. Nous allons voir comment ils sont résolus dans Manux, mais regardons d'abord un peu comment la pagination est mise en œuvre.

10.3.1 Pagination

Le mécanisme de pagination des processeurs Intel permet, comme l'illustre la figure 8, de construire une mémoire virtuelle, découpée en pages (de 4 Ko) à partir de pages physiques situées "n'importe où" en mémoire physique (ou même sur un autre support tel qu'un disque dur ou autre).

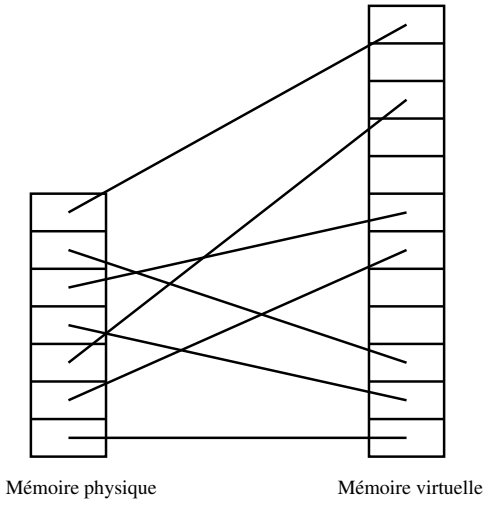

FIGURE 8 – Principe de la pagination

Le processeur offre (via sa MMU) des mécanismes évolués permettant la mise en place de la pagination. La pagination est activée en positionnant à un le bit 31 du registre CR0.

Lorsque la pagination est activée, le registre CR3 doit contenir l'adresse (physique) d'une page mémoire contenant le *répertoire de pagination* .

10.3.2 Gestion de la mémoire physique dans ManuX

La mémoire physique est partagée entre une partie système et une partie application. La première partie est dédiée à la gestion du système et la seconde et utilisée pour satisfaire les besoins en mémoire des différentes applications.

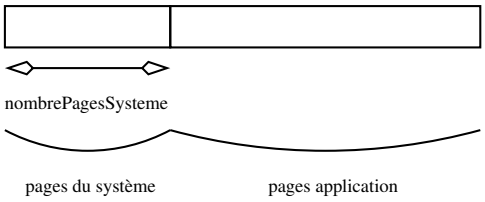

FIGURE 9 – Structure de la mémoire physique

Au niveau système, la mémoire est gérée par page (une gestion plus fine doit donc être implantée au niveau application, par le biais des librairies).

Organisation de la mémoire physique Traditionnellement, sur un PC, la mémoire physique est organisée comme décrit par la figure **??**.

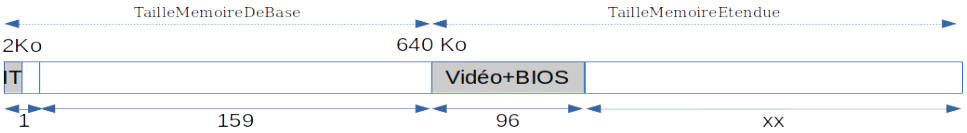

FIGURE 10 – État de la mémoire au démarage

Une partie est réservée pour le BIOS et la vidéo, nous nous garderons bien d'aller y lire ou écrire pour autre chose !

L'état de cette mémoire à la fin du boot (décrit plus haut) est donné par la figure **??** que nous redonnons ici.

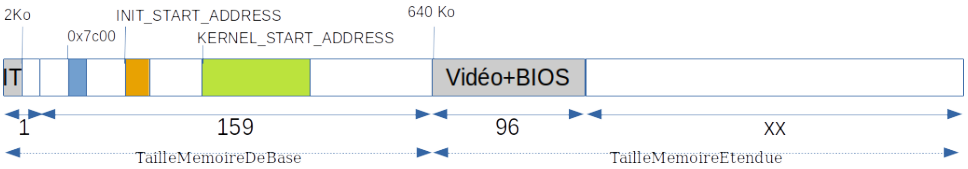

FIGURE 11 – État de la mémoire après le boot

Il est important de l'avoir en tête pour la suite afin de déterminer quelles sont les zones que nous pouvos allouer et celles qui ne doivent pas être utilisées.

Allocation de la mémoire Chaque page mémoire peut être allouée à une tâche (ou au noyau) ou libre (non allouée). C'est le tableau `proprietairePage` défini dans le fichier `noyau/memoire.c` qui définit cela comme illustré par la figure 12. Une page libre sera associée à un propriétaire nul et une page allouée au système à la tâche 1.

FIGURE 12 – Définition de l'allocation des pages mémoire

Initialisation

Allocation Deux fonctions permettent d'allouer une page mémoire, ce sont

```
void * allouerPageSysteme();
void * allouerPage();
```

qui permettent d'obtenir respectivement une page mémoire système et une page mémoire application.

10.3.3 Organisation de la mémoire virtuelle

11 La librairie du noyau

Je vais décrire ici rapidement les outils fournis pour le noyau, c'est-à-dire tout ce qui est codé dans le répertoire `lib`.

11.1 Le journal

Le journal permet d'archiver les messages émis par les différents éléments du noyau grâce à la fonction *`printk()`* .

Le journal est défini dans le fichier `include/journal.h` et implanté dans le fichier `lib/journal.c`.

11.2 Le ramdisk

Le but du ramdisk est uniquement de nous permettre de valider certains éléments du système, tels que le système de fichiers, sans avoir besoin d'éléments complexes qui seront bien plus longs à mettre en place et à tester, comme un pilote de disque dur ou de lecteur de disquettes.

Il permet de plus une certaine forme de communication avec le système d'exploitation de développement puisqu'une image disque peut être construite sur ce système avant d'être exploitée dans Manux-32.

12 Les outils de synchronisation

Les moyens de synchronisation entre tâches actuellement disponibles sont
— les opérations atomiques ;
— les exclusions mutuelles.

12.1 Opérations atomiques

Les opérations élémentaires de synchronisation sont définies dans le fichier `atomique.h`. Elles se basent sur le type *`Atomique`* et les opérations qui lui sont associées. A part l'initialisation et la consultation, l'opération importante est la suivante

```
booleen atomiqueTestInit(volatile Atomique * atom, uint32 val, uint32 cond);
```

Elle compare la valeur de l'`Atomique` avec `cond` et, en cas d'égalité, affecte l'`Atomique` avec la valeur `val` et renvoie `TRUE`, sinon elle ne fait rien et renvoie `FALSE`. Ce qui est important, bien sur, c'est que la comparaison et l'éventuelle affectation soient réalisées de façon *atomique*.

12.2 Exclusions mutuelles

Les variables d'exclusion mutuelle permettent d'assurer que l'accés à une ressource (typiquement une section de code critique) se fait de façon exclusive entre les tâches.

Le fichier `atomique.h` définit le type et les fonctions suivants

```
typedef struct _ExclusionMutuelle {
   Atomique   verrou;
   ListeTache tachesEnAttente;
} ExclusionMutuelle;

void initialiserExclusionMutuelle(ExclusionMutuelle * em);
void entrerExclusionMutuelle(ExclusionMutuelle * em);
void sortirExclusionMutuelle(ExclusionMutuelle * em);
```

Naturellement, comme leurs noms l'indiquent, les trois sous-programmes permettent respectivement d'initialiser une variable d'exclusion mutuelle, de prendre une exclusion mutuelle et de libérer une exclusion mutuelle.

13 L'ordonnancement des tâches

Le scheduler est l'entité chargée de la mise en œuvre de l'ordonnancement. Aussi étrange que cela puisse paraitre, il est implanté sous la forme d'un processus. En fait, chaque fois qu'il est nécessaire de changer de tâche active (lorsque le quantum de temps alloué à celle en cours est révolu, ou qu'elle se bloque sur une entrée/sortie), le système bascule vers la tâche scheduler, qui se charge alors d'élire la prochaine tâche et de l'activer.

Le corps du scheduler se résume alors à une boucle folle qui ne fait qu'appeler le sous-programme `basculerTache()` chargé de l'élection de la prochaine tâche.

Naturellement, la tâche de l'ordonnanceur n'est pas stockée dans la liste des tâches actives du système.

14 Le système de fichiers

A Les appels système de ManuX

Voici la liste des appels système de ManuX

A.1 creerNouvelleTache

A.2 basculerTache

A.3 ecrireConsole